

LABORATORIO DI PROGRAMMAZIONE
CORSO DI LAUREA IN SICUREZZA DEI SISTEMI E DELLE RETI
INFORMATICHE
UNIVERSITÀ DEGLI STUDI DI MILANO
2025–2026

INDICE

Parte 1. Ricorsione	3
Esercizio 1	3
<i>Lunghezza di una stringa: implementazione ricorsiva</i>	3
Tempo: 15 min.	3
Esercizio 2	3
<i>Occorrenze di un carattere in una stringa: implementazione ricorsiva</i>	3
Tempo: 15 min.	3
Esercizio 3	4
<i>Numeri di Catalan</i>	4
Tempo: 20 min.	4
Esercizio 4	4
<i>Tabulazione dei numeri di Catalan</i>	4
Tempo: 25 min.	4
Esercizio 5	5
<i>Numeri di Delannoy</i>	5
Tempo: 20 min.	5
Esercizio 6	5
<i>Tabulazione dei numeri di Delannoy</i>	5
Tempo: 25 min.	5
Esercizio 7	6
<i>Numeri di Bell</i>	6
Tempo: 25 min.	6
Esercizio 8	7
<i>Successioni maschio-femmina di Hofstadter</i>	7
Tempo: 30 min.	7
 Parte 2. Allocazione dinamica della memoria	 8
Esercizio 9	8
<i>Funzioni di base per gestire una lista</i>	8
Tempo: 60 min.	8
Esercizio 10	10
<i>Funzione main e esercizi semplici</i>	10
Tempo: 60 min.	10
Esercizio 11	12

Tratti dagli esercizi del corso del prof. Vincenzo Marra.
Ultima revisione: 23 novembre 2025.

Array

12

Tempo: 60 min.

12

Parte 1. Ricorsione

ESERCIZIO 1

Lunghezza di una stringa: implementazione ricorsiva.

Tempo: 15 min.

Scrivete una funzione di prototipo `int lung(char *)` che accetti in ingresso una stringa e ne restituisca la lunghezza. L'implementazione della funzione deve essere ricorsiva. Per convenzione, se il puntatore ricevuto dalla funzione è `NULL`, la lunghezza restituita è -1. Se lunghezza della stringa vuota—cioè, di un array di `char` il cui primo carattere sia `\0`—è zero. Scrivete una funzione `main` che vi permetta di testare la vostra implementazione della funzione `lung`.

Suggerimento. La lunghezza di una stringa della forma `cs`, dove `c` è un carattere ed `s` una stringa, è pari alla lunghezza di `s` più 1.

La codifica iterativa della funzione per il calcolo della lunghezza di una stringa, che avete già scritto in esercizi precedenti, è semplice come la versione ricorsiva e risulta molto più efficiente in esecuzione. L'Esercizio 1 ha quindi il solo scopo di farvi prendere dimestichezza con le implementazioni ricorsive. Ciò vale anche per altri esercizi di questa lezione.

ESERCIZIO 2

Occorrenze di un carattere in una stringa: implementazione ricorsiva.

Tempo: 15 min.

Scrivete una funzione di prototipo `int occ(char *, char)` che accetti in ingresso una stringa e un carattere, e restituisca il numero di occorrenze del carattere nella stringa. L'implementazione della funzione deve essere ricorsiva. Per convenzione, se il puntatore ricevuto dalla funzione è `NULL`, il numero di occorrenze di qualunque carattere nella stringa è -1. Scrivete una funzione `main` che vi permetta di testare la vostra implementazione della funzione `occ`.

I numeri di Catalan

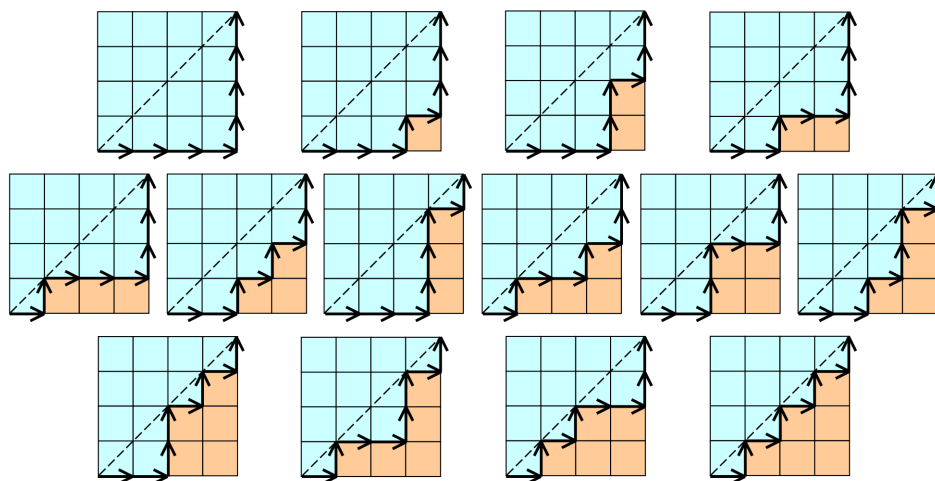
I *numeri di Catalan* prendono il nome dal matematico belga Eugène Charles Catalan (1814–1894). Essi si denotano con $C(n)$, dove n è un intero ≥ 0 , e contano il numero di cammini in una griglia quadrata di dimensione $n \times n$ che partono dall'angolo sud-ovest e arrivano all'angolo nord-est senza mai oltrepassare la diagonale sud-ovest/nord-est della griglia, impiegando solo passi verso nord ed est. Si veda la Figura 1.

I numeri di Catalan soddisfano la relazione ricorsiva:

$$C(n) = \frac{2(2n-1)C(n-1)}{n+1}$$

con condizione iniziale

$$C(0) = 1.$$

FIGURA 1. I 14 cammini di Catalan sulla griglia 4×4 .

ESERCIZIO 3

Numeri di Catalan.

Tempo: 20 min.

Scrivete un programma ricorsivo che legga in ingresso un intero $n \geq 0$ e calcoli e visualizzi il numero di Catalan $C(n)$. Nel caso in cui l'utente inserisca un valore che non soddisfa la condizione $n \geq 0$, forzate il reinserimento.

ESERCIZIO 4

Tabulazione dei numeri di Catalan.

Tempo: 25 min.

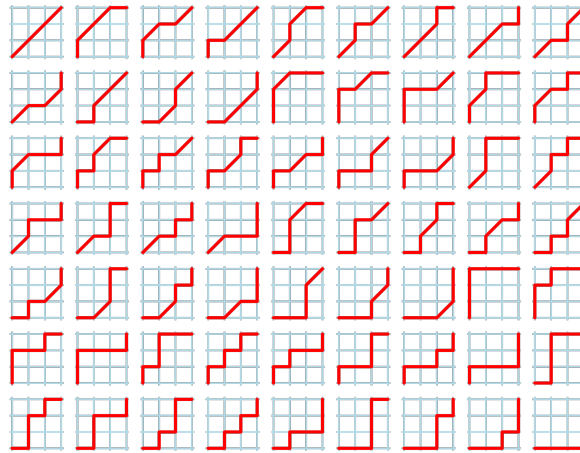
Scrivete un programma che legga in ingresso un intero $n \geq 0$ e calcoli e visualizzi la sequenza $C(0), C(1), \dots, C(n)$ di numeri di Catalan. Per esempio, nel caso in cui $n = 9$ l'output del vostro programma dovrebbe essere:

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862.

Riutilizzate la funzione ricorsiva scritta per risolvere l'Esercizio 3.

I numeri di Delannoy

I *numeri di Delannoy* prendono il nome dall'ufficiale francese Henri-Auguste Delannoy (1833–1915), matematico dilettante. Essi si denotano con $D(m, n)$, dove m ed n sono interi ≥ 0 , e contano il numero di cammini in una griglia rettangolare di dimensione $m \times n$ che partono dall'angolo sud-ovest e arrivano all'angolo nord-est, impiegando solo passi verso nord, est e nord-ovest. Si veda la Figura 2.

FIGURA 2. I 63 cammini di Delannoy sulla griglia 3×3 .

I numeri di Delannoy soddisfano la relazione ricorsiva:

$$D(m, n) = D(m-1, n) + D(m, n-1) + D(m-1, n-1) \quad (1)$$

con condizioni iniziali

$$D(0, n) = D(m, 0) = 1. \quad (2)$$

ESERCIZIO 5

Numeri di Delannoy.

Tempo: 20 min.

Scrivete un programma ricorsivo che legga in ingresso due interi $m, n \geq 0$ e calcoli e visualizzi il numero di Delannoy $D(m, n)$. Nel caso in cui l'utente inserisca valori che non soddisfano la condizione $m, n \geq 0$, forzate il reinserimento.

ESERCIZIO 6

Tabulazione dei numeri di Delannoy.

Tempo: 25 min.

Scrivete un programma che legga in ingresso due interi $m, n \geq 0$ e calcoli e visualizzi una tabella rettangolare di dimensioni $m \times n$ il cui elemento di posto (i, j) sia il numero di Delannoy $D(i, j)$. Per esempio, nel caso in cui $m = n = 9$ l'output del vostro programma dovrebbe collimare con quello mostrato in Figura 3. Riutilizzate la funzione ricorsiva scritta per risolvere l'Esercizio 5.

I numeri di Bell

1	1	1	1	1	1	1	1	1	1
1	3	5	7	9	11	13	15	17	19
1	5	13	25	41	61	85	113	145	181
1	7	25	63	129	231	377	575	833	1159
1	9	41	129	321	681	1289	2241	3649	5641
1	11	61	231	681	1683	3653	7183	13073	22363
1	13	85	377	1289	3653	8989	19825	40081	75517
1	15	113	575	2241	7183	19825	48639	108545	224143
1	17	145	833	3649	13073	40081	108545	265729	598417
1	19	181	1159	5641	22363	75517	224143	598417	1462563

FIGURA 3. I numeri di Delannoy $D(i, j)$ con $i, j \in \{0, 1, \dots, 9\}$.

I *numeri di Bell* prendono il nome dallo storico della matematica statunitense di origini scozzesi Eric Temple Bell (1883–1960). Essi si denotano con $B(n)$, dove n è un intero ≥ 0 . Per definizione, $B(n)$ è il numero di partizioni di un insieme di cardinalità n .

I numeri di Bell soddisfano la relazione ricorsiva

$$B(n) = \sum_{k=0}^{n-1} \binom{n-1}{k} B(k) \quad (\dagger)$$

per $n \geq 1$, con condizioni iniziali

$$B(0) = B(1) = 1.$$

Nella $(\dagger)$, $\binom{n-1}{k}$ è il *coefficiente binomiale*, per definizione pari al numero di sottoinsiemi di cardinalità k di un insieme di $n-1$ elementi, per $n \geq 1$. Il coefficiente binomiale soddisfa l'identità

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad (*)$$

per $n \geq k \geq 0$. Nella $(*)$, $n!$ denota il *fattoriale* di n , per definizione pari al numero delle permutazioni di un insieme di n elementi. Esso soddisfa la relazione ricorsiva

$$n! = n((n-1)!)$$

per $n \geq 1$, con condizione iniziale

$$0! = 1.$$

Si veda la Figura 4.

ESERCIZIO 7

Numeri di Bell.

Tempo: 25 min.

Scrivete una funzione `int bell(int n)` che, dato in ingresso un numero naturale $n \geq 0$, restituisca il numero di Bell $B(n)$. Per implementare `int bell(int n)` ricorsivamente, scrivete due funzioni ausiliarie. La prima, `int fatt(int n)`, restituisce il fattoriale del numero naturale in ingresso $n \geq 0$, ed è implementata ricorsivamente. La seconda, `int binom(n,k)`, restituisce il coefficiente binomiale $\binom{n}{k}$, dati due naturali in ingresso $n \geq k \geq 0$, ed è implementata usando `int`

n	0	1	2	3	4	5	6	7	8	9
$B(n)$	1	1	2	5	15	52	203	877	4140	21147

n	0	1	2	3	4	5	6	7	8	9
$n!$	1	1	2	6	24	120	720	5040	40320	362880

n	$\binom{n}{0}$	$\binom{n}{1}$	$\binom{n}{2}$	$\binom{n}{3}$	$\binom{n}{4}$	$\binom{n}{5}$	$\binom{n}{6}$	$\binom{n}{7}$
0	1							
1	1	1						
2	1	2	1					
3	1	3	3	1				
4	1	4	6	4	1			
5	1	5	10	10	5	1		
6	1	6	15	20	15	6	1	
7	1	7	21	35	35	21	7	1

FIGURA 4. Tabulazione dei numeri di Bell, del fattoriale e del coefficiente binomiale.

`fatt(int n)`. Scrivete una procedura `main` che chieda all'utente di inserire un intero $n \geq 0$ e visualizzi sul terminale il numero di Bell $B(n)$. Forzate il reinserimento se l'intero n digitato dall'utente non soddisfa la condizione $n \geq 0$.

ESERCIZIO 8

Successioni maschio-femmina di Hofstadter.

Tempo: 30 min.

Si consideri la coppia di successioni definite da

$$F(n) = n - M(F(n-1)), \quad (3)$$

$$M(n) = n - F(M(n-1)), \quad (4)$$

con valori iniziali

$$F(0) = 1, \quad (5)$$

$$M(0) = 0. \quad (6)$$

Si tratta delle *successioni maschio-femmina di Hofstadter*. Per maggiori informazioni si veda, ad esempio, [questa](#) pagina di Wolfram's MatWorld.

I primi dieci numeri della sequenza maschio $M(n)$ sono 0, 0, 1, 2, 2, 3, 4, 4, 5, 6. I primi dieci numeri della sequenza femmina $F(n)$ sono 1, 1, 2, 2, 3, 3, 4, 5, 5, 6.

Si implementi, secondo le seguenti specifiche, un programma per calcolare i primi $n+1$ numeri delle sequenze $F(n)$ e $M(n)$, per un valore n inserito dall'utente.

- (a) Il programma chiede all'utente di inserire un numero intero $n \geq 0$, verificando la correttezza dell'input. Se $n < 0$ il programma forza il reinserimento.

- (b) Il programma visualizza in output i primi $n + 1$ numeri delle sequenze $F(n)$ e $M(n)$. Ad esempio, per $n = 4$ l'output del programma sarà:

n	$F(n)$	$M(n)$
0	1	0
1	1	0
2	2	1
3	2	2
4	3	2

Parte 2. Allocazione dinamica della memoria

ESERCIZIO 9

Funzioni di base per gestire una lista.

Tempo: 60 min.



FIGURA 5. Esempio di una lista contenente tre valori. Le frecce indicano che un nodo contiene un riferimento a un altro elemento della lista, mentre il simbolo $\perp$ rappresenta il valore NULL.

Questo è un esempio di un esercizio tratto da un tema di esame.

Una **lista** è una struttura dati *dinamica* composta da un insieme di elementi detti **nodi** collegati linearmente tra loro. Ogni nodo non è altro che un contenitore di dati. Nel nostro caso ogni nodo conterrà un valore numerico intero (**int**).

All'interno di una lista, ogni nodo contiene un riferimento (un *puntatore*) al nodo successivo (**next**), mentre la lista terrà un riferimento al primo nodo (**header**). Una lista vuota è un puntatore a NULL.

Dovete implementare una **lista** e le funzioni che gestiscono come vengono inseriti e rimossi dei dati da essa. L'implementazione dovrà supportare diverse modalità di inserimento e di rimozione degli elementi. La lista dovrà permettere all'utente di effettuare le seguenti operazioni:

- **inserisciInTesta()**: inserisce un nuovo elemento nella lista in prima posizione, in testa alla lista;
- **inserisciInCoda()**: inserisce un nuovo elemento nella lista in ultima posizione, in coda alla lista;
- **rimuoviInTesta()**: rimuove il primo elemento della lista, la testa;
- **rimuoviInCoda()**: rimuove l'ultimo elemento della lista, la coda.

Per inserire un nuovo elemento nella lista, dovrete creare un nuovo *nodo* in maniera dinamica. Un esempio di una lista contenente 3 valori è visibile in Figura 1. Alla fine della Parte 2 trovate un esempio di utilizzo della lista.

I primi esercizi serviranno per implementare i metodi e le strutture dati che utilizzerete durante la restante parte dell'esame, per la prova di C. Testate con dei casi semplici e con un main (anche senza usare le `printf` e `scanf` le funzioni ogni volta che ne finite una. Per testare una funzione, bisogna valutarla

- su di un caso normale;
- su casi limite (ad esempio, lista vuota);
- su casi di errore (ad esempio, se l'utente prova ad eliminare un elemento in una lista vuota);

Potete utilizzare le funzioni che avete già scritto e di cui avete verificato la correttezza per testare le funzioni implementate successivamente.

Esercizio 9.1

Definite una struttura `Nodo` che rappresenta un nodo della lista. La struttura contiene un campo per il *valore* dell'elemento, e un campo che si riferisce al nodo successivo. Il valore contenuto da un nodo è un numero intero.

Esercizio 9.2

Scrivete una funzione `inserisciInTesta()` che prende in input un valore intero e lo inserisce in testa alla lista. La funzione restituisce il valore dell'elemento inserito, `-1` in caso di errore.

Esercizio 9.3

Scrivete una funzione `inserisciInCoda()` che prende in input un valore intero e lo inserisce in coda alla lista. La funzione restituisce il valore dell'elemento inserito, `-1` in caso di errore.

Esercizio 9.4

Scrivete una funzione `rimuoviInTesta()` che elimina il primo valore della lista. Restituisce `-1` se la lista è vuota.

Esercizio 9.5

Scrivete una funzione `rimuoviInCoda()` che elimina l'ultimo elemento della lista. Restituisce `-1` se la lista è vuota.

Esercizio 9.6

Scrivete una funzione `cancella()` che, ricevuto una lista, ne cancelli ogni suo elemento, svuotandola.

Attenzione alla gestione della memoria allocata dinamicamente.

Quando trovate una indicazione che dovete prestare attenzione a qualcosa, significa che dovete farlo. In questo caso, la nota vi suggerisce che dovrete ricordarvi sempre che una `malloc` abbia avuto successo, e dovrete fare una `free` ogni volta che una porzione di memoria non è più necessaria.

Il main che utilizza la struttura dati così definita è oggetto della Parte 2. Testate le singole funzioni, e considerate i possibili errori che potrebbero verificarsi durante l'esecuzione.

ESERCIZIO 10

*Funzione **main** e esercizi semplici.*

Tempo: 60 min.

Dovete ora implementare delle funzioni semplici che operano sulla lista. Leggete bene la consegna perché spesso alcune funzioni sono utili anche a voi per debuggare il vostro codice; partite da sviluppare quelle funzioni, e poi potrete riutilizzarle. Ad esempio, in questo caso, la funzione **stampa** vi è molto utile.

Esercizio 10.1 Implementate una funzione **salva()**, che riceve come input una lista, definita come in Esercizio 9, e un nome di file. La funzione scrive i valori della lista nel file, uno per volta.

Esercizio 10.2 Implementate una funzione **carica()**, che riceve come input un nome di file e legge da esso un numero, non noto a priori, di interi. Tali valori verranno inseriti in testa in una lista definita come nell'Esercizio 9. In caso il file esista, assumete che sia strutturato in maniera corretta.

Esercizio 10.3 Implementate una funzione **stampa()** che stampa a schermo il contenuto della lista, in ordine.

Esercizio 10.4 Scrivete una funzione **rimuovi()** che elimina un elemento dalla lista, dato il suo valore. In caso l'elemento non sia presente nella lista, la funzione ritornerà -1 . Nel caso vi siano più occorrenze dell'elemento nella lista, solo una verrà eliminata. Nel caso l'elemento sia presente, la funzione restituisce il valore dell'elemento eliminato.

Gestire le eccezioni.

Esercizio 10.5 Implementa una funzione ricorsiva `stampaRicorsiva` che stampi a schermo gli elementi della lista in ordine inverso. Utilizzate la **ricorsione** per ottenere questo risultato. Ad esempio, se la lista contiene 19, 12, 37, 22 a schermo dovrà essere stampato 22, 37, 12, 19.

Esercizio 10.6

Il `main` è una collezione delle funzioni che avete sviluppato. Potete farlo mano a mano che sviluppate le singole funzioni.

Scrivere un `main()` che permetta all'utente di utilizzare una lista. Il `main()` chiederà un valore numerico all'utente e si comporterà nel seguente modo:

- 1 Aggiungi un elemento in testa (inserito dall'utente)
- 2 Rimuovi un elemento dalla testa
- 3 Aggiungi un elemento in coda (inserito dall'utente)
- 4 Rimuovi un elemento dalla coda
- 5 Rimuovi un elemento in lista (inserito dall'utente)
- 6 Carica lista da file
- 7 Salva lista su file
- 8 Stampa
- 9 Stampa la lista in ordine inverso
- 10 Cancella gli elementi di una lista.
- 0 Esci dal programma

Il programma continuerà a chiedere una nuova istruzione **valida** all'utente in maniera continuata, fino all'inserimento del valore di uscita.

ESEMPIO:

Leggete tutto il testo della consegna, perché l'esempio che viene fornito in fondo può aiutarvi a capire meglio il testo.

```

Lista: NULL
inserisciInTesta(1)
Lista: 1 → NULL
inserisciInTesta(3)
Lista: 3 → 1 → NULL
inserisciInCoda(8)
Lista: 3 → 1 → 8 → NULL
inserisciInCoda(2)
Lista: 3 → 1 → 8 → 2 → NULL
inserisciInTesta(10)
Lista: 10 → 3 → 1 → 8 → 2 → NULL
rimuoviInTesta()
Lista: 3 → 1 → 8 → 2 → NULL
rimuovi(3)
Lista: 1 → 8 → 2 → NULL

```

ESERCIZIO 11

Array.

Tempo: 60 min.

Questo è un altro esempio di un esercizio tratto da un tema di esame, in questo caso una variante di quello implementato in precedenza

Un **array** è una struttura dati *sequenziale* e di dimensione prefissata composta da un insieme di organizzato di oggetti sequenziali tra loro. Nel nostro caso ogni elemento dell'array conterrà una *stringa*; ogni stringa sarà di una singola parola (senza spazi); la dimensione del singolo elemento dell'array, della singola stringa, non è nota a priori.

In questo esercizio dovrete implementare un **array** di stringhe e le funzioni che gestiscono come vengono inseriti e rimossi dei dati da esso. Gli array conterranno al massimo **100** elementi (parole); tale dimensione è definita a compile time. Inizialmente saranno vuoti, e l'inserimento di un nuovo dato sarà fatto nella prima posizione libera dell'array. Dovrete implementare le funzioni per effettuare le seguenti operazioni:

- **inserisci**: inserisce un nuovo elemento nell'array nella prima posizione libera;
- **rimuovi**: rimuove l'ultimo elemento inserito nell'array;
- **unisci**: riceve due array; da questi ne crea un terzo; in questo array sarà contenuto prima l'array 1, poi l'array 2.

Per inserire un nuovo elemento nell'array, dovrete allocare una nuova *stringa* in maniera dinamica. Alla fine della Parte 2 trovate un esempio di utilizzo degli array.

Esercizio 11.1

Definite una struttura **ArrayParole**. La struttura contiene un campo con l'*array* di stringhe, di dimensione 100, e un contatore che tiene traccia di quante parole sono state effettivamente inserite nell'array. La lunghezza di ogni elemento dell'array, ovvero di ogni stringa contenente una parola, non è nota a priori; quando un nuovo elemento viene inserito, viene quindi allocata dinamicamente.

Esercizio 11.2

Scrivete una funzione **inserisci** che riceve come input una stringa e la inserisce nella prima posizione disponibile dell'array. La funzione restituisce quanti elementi sono stati inseriti nell'array in caso positivo, -1 in caso di errore.

Esercizio 11.3

Scrivete una funzione **rimuovi** che elimina l'ultimo elemento dell'array.

Esercizio 11.4

Scrivete una funzione **unisci** che, ricevute due **ArrayParole**, crea una terza struttura composta dalle parole della prima concatenate alle parole della seconda. Il nuovo array deve contenere delle copie delle stringhe. La grandezza del nuovo array è sempre 100; assumete che i due array da unire abbiano, complessivamente, meno di 100 elementi.

Ad esempio, se il primo array contiene le parole:

```
A1[0]: "Corso"; A1[1]: "di"; A1[2]: "Informatica";
```

e il secondo array contiene le parole:

```
A2[0]: "Esame"; A2[1]: "di"; A2[2]: "Programmazione";
```

la struttura dati risultante risultante dalla loro unione sarà:

```
A3[0]: "Corso"; A3[1]: "di"; A3[2]: "Informatica"; A3[3]: "Esame";
```

```
A3[4]: "di"; A3[5]: "Programmazione";
```

Esercizio 11.5

Scrivete una funzione **cancella** che, ricevuto un array, ne cancelli ogni suo elemento, svuotandolo.

Attenzione alla gestione della memoria allocata dinamicamente.

Testate le singole funzioni, e considerate i possibili errori che potrebbero verificarsi durante l'esecuzione.

Anche in questo caso, la seconda parte del tema di esame vi chiedeva di implementare un **main** e le funzioni per stampare, salvare, caricare gli array. Gli esercizi sono simili a quelli della Parte 2, vi lasciamo come riferimento l'esercizio sulla ricorsione.

Esercizio 11.6

Implementa una funzione ricorsiva **Ribalta** che inverta gli elementi dell'array, facendo diventare il primo elemento l'ultimo, il secondo il penultimo, e viceversa. Utilizzate la **ricorsione** per ottenere questo risultato. Ad esempio, se l'array contiene **piove, che, ore, sono**, l'array ribaltato diventa **sono ore che piove**. La modifica viene fatta sull'array stesso.

ESEMPIO:

```
Array 1: lunghezza:0;
```

```
Array 2: lunghezza:0;
```

```
Array 3: lunghezza:0;
```

```
Inserisci in Array 1: "Lunedì"
```

```
Array 1: [0]:Lunedì; lunghezza:1;
```

```
Array 2: lunghezza:0;
```

```
Array 3: lunghezza:0;
```

```
Inserisci in Array 2: "Mercoledì";
```

```
Array 1: [0]:"Lunedì"; lunghezza:1;
```

```
Array 2: [0]:"Mercoledì"; lunghezza:1;
```

```
Array 3: lunghezza:0
```

```
Inserisci in Array 1: Martedì;
```

```
Array 1: [0]:"Lunedì"; [1]:"Martedì"; lunghezza:2;
```

```
Array 2: [0]:"Mercoledì"; lunghezza:1;
```

```
Array 3: lunghezza:0
```

```
Inserisci in Array 2: "Giovedì";
```

```
Array 1: [0]:"Lunedì"; [1]:"Martedì"; lunghezza:2;
```

```
Array 2: [0]:"Mercoledì" [1]:"Giovedì"; lunghezza:2;
```

```
Array 3: lunghezza:0
```

```
Unisci gli Array:
```

```
Array 1: [0]:"Lunedì"; [1]:"Martedì"; lunghezza:2;  
Array 2: [0]:"Mercoledì" [1]:"Giovedì"; lunghezza:2;  
Array 3: [0]:"Lunedì"; [1]:"Martedì"; [2]:"Mercoledì" [3]:"Giovedì"; lunghezza:4;  
Rimuovi da Array 1;  
Array 1: [0]:"Lunedì"; lunghezza:1;  
Array 2: [0]:"Mercoledì" [1]:"Giovedì"; lunghezza:2;  
Array 3: [0]:"Lunedì"; [1]:"Martedì"; [2]:"Mercoledì" [3]:"Giovedì"; lunghezza:4;
```

DIPARTIMENTO DI INFORMATICA, UNIVERSITÀ DEGLI STUDI DI MILANO,